

Software Requirements Specification (SRS)

English Phonics App

Document Version: 1.0

Product Version: 0.1.0

Date: February 2025

Status: Draft

Table of Contents

1. Introduction
 2. Overall Description
 3. Functional Requirements
 4. External Interface Requirements
 5. Non-Functional Requirements
 6. System Attributes
 7. Appendices
-

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the **English Phonics App**—a multi-platform system for learning English phonics. The document is intended for developers, testers, project managers, and stakeholders. It defines what the system shall do (functional requirements) and how it shall behave (non-functional requirements), and serves as a contract for development and acceptance.

1.2 Scope

Product name: English Phonics App

Scope:

- **Backend:** A REST API that provides authentication, user management, phoneme and exercise content, pronunciation assessment, progress tracking, social features (friends), and teacher–student associations.

- **Web application (phonics-web):** A browser-based client for students and teachers to register, log in, view lessons, complete exercises with pronunciation practice, and see progress and recommendations.
- **Mobile application:** A native or cross-platform mobile client (to be developed) offering core learning flows aligned with the web and backend.

Out of scope for this SRS:

- Content authoring tools external to the defined admin APIs
- Offline-first mobile behavior (may be added in a later revision)
- Integration with third-party LMS or school information systems

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
Phoneme	Smallest unit of sound in a language; in this system, a phonetic symbol with optional audio and metadata.
Exercise	A single practice item (e.g. word, sentence, or phoneme) that a user can listen to and repeat, with optional pronunciation scoring.
Lesson	A grouping of exercises and phonemes, typically associated with a level (e.g. LEVEL1–LEVEL5).
Progress	Record of a user’s completion and scores for lessons/exercises.
JWT	JSON Web Token; used for API authentication.
RBAC	Role-Based Access Control.
TTS	Text-to-Speech.
SRS	Software Requirements Specification.
API	Application Programming Interface.
CRUD	Create, Read, Update, Delete.

1.4 References

ID	Document / Source
REF-1	Project repository: English Phonics App (backend, phonics-web)
REF-2	DOCUMENTATION.md – Current implementation and expected work
REF-3	APP_SUMMARY.md – High-level summary of features and structure

1.5 Overview

The rest of this SRS is organized as follows:

- **Section 2** – Overall description of the product, users, and environment.
 - **Section 3** – Detailed functional requirements (system features).
 - **Section 4** – External interfaces (user, hardware, software, communication).
 - **Section 5** – Non-functional requirements (performance, security, etc.).
 - **Section 6** – System attributes (reliability, maintainability).
 - **Section 7** – Appendices (e.g. API summary, environment variables).
-

2. Overall Description

2.1 Product Perspective

The English Phonics App is a standalone product composed of:

- **Backend server:** REST API (FastAPI), PostgreSQL database, optional Azure Cognitive Services (Speech, TTS). It can be deployed via Docker and consumed by web and mobile clients.
- **Web client:** Next.js application (phonics-web) running in modern browsers.
- **Mobile client:** To be implemented; will consume the same API.

The system integrates with:

- **Azure Cognitive Services (Speech):** For pronunciation assessment.
- **Azure TTS (or similar):** For generating reference audio when pre-recorded audio is not available.
- **PostgreSQL:** As the primary data store.

2.2 Product Functions (High-Level)

- **User management:** Registration, login, profile, roles (Student, Teacher, Admin).
- **Authentication and authorization:** JWT-based API auth; RBAC for endpoints.

- **Phonemes:** Manage phonetic symbols and audio; list and retrieve for learning.
- **Lessons and exercises:** Structure curriculum; serve exercises with reference audio and pronunciation submission.
- **Pronunciation assessment:** Accept user audio, return score and feedback (e.g. accuracy, fluency).
- **Progress and recommendations:** Track completion and scores; recommend next exercises based on weak phonemes and uncompleted lessons.
- **Social (friends):** Send/accept friend requests; list friends; get recommendations (e.g. by school, city, grade).
- **Teacher–student associations:** Link teachers to students; list “my students” / “my teachers.”
- **Gamification (planned):** Achievements and badges driven by progress/scores.

2.3 User Classes and Characteristics

User class	Description	Typical use
Student	Learner (e.g. child or adult)	Register, log in, complete lessons/exercises, record pronunciation, view progress and recommendations, manage friends.
Teacher	Educator	Log in, view/assign students via associations, optionally view student progress.
Administrator	System manager	Full user management, phoneme CRUD, future lesson/word management, system configuration.
Guest / Anonymous	Unauthenticated visitor	View public content (e.g. list phonemes, get exercise/reference audio); must register/login for progress and submission.

2.4 Operating Environment

Backend

- **Runtime:** Python 3.10+ (recommended 3.12).
- **Server:** Uvicorn (or equivalent ASGI server).
- **Database:** PostgreSQL 12+.
- **OS:** Linux recommended; Windows/macOS supported for development.
- **External services:** Azure Speech (and optionally TTS) when pronunciation/audio features are used.

Web

- **Runtime:** Node.js 18+ (LTS).
- **Browsers:** Current versions of Chrome, Firefox, Safari, Edge (evergreen).
- **Devices:** Desktop and tablet; responsive layout for smaller screens.

Mobile (planned)

- **Platforms:** iOS and/or Android (to be decided).
- **OS versions:** To be specified at project start.

2.5 Design and Implementation Constraints

- **API:** REST over HTTP/HTTPS; JSON request/response.
- **Authentication:** JWT in `Authorization: Bearer <token>` header.
- **Password storage:** Argon2 (or similarly strong) hashing only; no plain-text passwords.
- **Audio formats:** Backend accepts common formats (e.g. MP3, WAV, OGG, WebM, M4A) for uploads; specific limits to be set (e.g. max file size, max duration).
- **Database:** Relational model (PostgreSQL); migrations via Alembic.
- **Web:** Next.js App Router; TypeScript for type safety.

2.6 Assumptions and Dependencies

Assumptions

- Users have a stable internet connection for API and (when used) Azure services.
- Teachers and students are pre-registered or self-register; no automatic import from external school systems.
- Content (phonemes, lessons, words) is managed via the system (API/admin) rather than external files only.
- Azure Speech (and TTS) subscriptions and keys are available where pronunciation/audio features are required.
- One production database is used; read replicas or sharding are not required in initial scope.

Dependencies

- PostgreSQL availability and backup.
- Azure Cognitive Services availability and quotas.
- Correct configuration of environment variables (database URL, secrets, Azure keys).
- Web clients support cookies/local storage for session (e.g. NextAuth).

3. Functional Requirements

3.1 Authentication and User Management

ID	Requirement	Priority	Status
FR-1.1	The system shall allow a user to register with name, email, password, and optional profile fields (e.g. age_group, grade_level, school_name).	High	Partial (backend: POST /users/; dedicated register endpoint planned)
FR-1.2	The system shall allow a user to log in with email and password and shall return a JWT access token.	High	Implemented

ID	Requirement	Priority	Status
FR-1.3	The system shall enforce a minimum password length (e.g. 8 characters) and store only a hashed form (e.g. Argon2).	High	Implemented
FR-1.4	The system shall support roles: Student, Teacher, Admin. Access to endpoints shall be restricted by role (RBAC).	High	Implemented
FR-1.5	The system shall allow an authenticated user to update their own profile (e.g. name, email) via a dedicated “me” endpoint.	High	Implemented
FR-1.6	The system shall allow an admin to list users with pagination and to soft-deactivate (or delete) users.	Medium	Implemented
FR-1.7	The system shall support password reset (request and reset token).	Medium	Planned
FR-1.8	The system may support email verification for new accounts.	Low	Planned

3.2 Phonemes and Content

ID	Requirement	Priority	Status
FR-2.1	The system shall allow an admin to create, read, update, and delete phonemes (symbol, description, type, lesson_id, audio).	High	Implemented
FR-2.2	The system shall accept audio file uploads for phonemes (e.g. MP3, WAV, OGG, WebM, M4A) and store them (e.g. local or cloud).	High	Implemented (local)

ID	Requirement	Priority	Status
FR-2.3	The system shall allow anyone (including unauthenticated) to list and retrieve phonemes (read-only).	High	Implemented
FR-2.4	The system shall support phoneme types (e.g. alphabet, long_vowel, short_vowel, consonant_blend, etc.) as defined in the data model.	Medium	Implemented
FR-2.5	The system shall allow CRUD operations for lessons (create, list by level/order, update, delete).	High	Planned
FR-2.6	The system shall allow CRUD operations for words and associate them with phonemes and exercises.	High	Planned

3.3 Exercises and Pronunciation

ID	Requirement	Priority	Status
FR-3.1	The system shall allow retrieval of an exercise by ID (content, type, linked phonemes/word, etc.).	High	Implemented
FR-3.2	The system shall provide reference audio for an exercise (pre-recorded phoneme/word audio or TTS-generated).	High	Implemented
FR-3.3	The system shall accept a user's pronunciation recording (audio file) for an exercise and return a score (e.g. 0–100) and feedback (e.g. accuracy, fluency, transcription).	High	Implemented

ID	Requirement	Priority	Status
FR-3.4	The system shall persist pronunciation scores per user and exercise and update progress (e.g. completed flag, best score, attempts).	High	Implemented
FR-3.5	Pronunciation assessment shall use a defined reference (e.g. phoneme symbol or word text) and a supported speech service (e.g. Azure Speech).	High	Implemented

3.4 Progress and Recommendations

ID	Requirement	Priority	Status
FR-4.1	The system shall record progress per user per lesson/exercise (e.g. completed, score, attempts, updated_at).	High	Implemented
FR-4.2	The system shall provide a list of recommended exercises for the current user (e.g. based on weak phonemes, then next uncompleted lesson, then fallback).	High	Implemented
FR-4.3	The system shall provide personalized motivational feedback based on recent pronunciation scores.	Medium	Implemented

3.5 Friends (Social)

ID	Requirement	Priority	Status
FR-5.1	A student shall be able to send a friend request to another user by ID.	High	Implemented

ID	Requirement	Priority	Status
FR-5.2	A student shall be able to accept or reject pending friend requests.	High	Implemented
FR-5.3	A student shall be able to list pending requests and list current friends.	High	Implemented
FR-5.4	The system shall provide friend recommendations (e.g. by school, city, grade_level).	Medium	Implemented

3.6 Teacher–Student Associations

ID	Requirement	Priority	Status
FR-6.1	A teacher or admin shall be able to create an association between a teacher and a student.	High	Implemented
FR-6.2	A teacher or admin shall be able to remove an association.	High	Implemented
FR-6.3	A teacher shall be able to list students linked to them.	High	Implemented
FR-6.4	A student shall be able to list teachers linked to them.	High	Implemented

3.7 Gamification and Achievements

ID	Requirement	Priority	Status
FR-7.1	The system shall store gamification items (e.g. name, description, points_required, image_url).	Medium	Model only
FR-7.2	The system shall record user achievements (user_id, gamification_id, achieved_at).	Medium	Model only

ID	Requirement	Priority	Status
FR-7.3	The system shall expose APIs to list achievements and a user's earned achievements, and to award achievements based on rules (e.g. score thresholds).	Medium	Planned

3.8 Web Application (phonics-web)

ID	Requirement	Priority	Status
FR-W1	The web app shall provide a login page (email, password) and authenticate via the backend login API, then maintain a session (e.g. NextAuth JWT).	High	Implemented
FR-W2	The web app shall provide a registration page and call the backend to create a user (and optionally log in).	High	Implemented
FR-W3	The web app shall protect routes that require authentication (e.g. dashboard, exercises) and use the stored access token for API calls.	High	Partial
FR-W4	The web app shall provide a lesson-style page with a consistent layout (e.g. BookPageLayout) and placeholders for "listen and repeat" content.	Medium	Implemented
FR-W5	The web app shall provide a dashboard showing progress summary and recommended exercises (when backend and UI are ready).	High	Planned

ID	Requirement	Priority	Status
FR-W6	The web app shall allow the user to play reference audio, record pronunciation, submit to the backend, and display score and feedback.	High	Planned
FR-W7	The web app shall allow the user to view and edit their profile (aligned with PUT /users/me/).	Medium	Planned
FR-W8	The web app shall allow students to use friend features (send request, accept/reject, list friends, recommendations).	Medium	Planned

3.9 Mobile Application

ID	Requirement	Priority	Status
FR-M1	The mobile app shall provide login and registration using the same backend auth APIs.	High	Planned
FR-M2	The mobile app shall securely store the access token (e.g. Keychain/Keystore).	High	Planned
FR-M3	The mobile app shall display lessons and exercises and allow playback of reference audio and submission of pronunciation.	High	Planned
FR-M4	The mobile app shall display progress and recommendations (same backend endpoints as web).	High	Planned
FR-M5	The mobile app shall allow profile view and edit.	Medium	Planned

4. External Interface Requirements

4.1 User Interfaces

- **Web:** Browser-based UI; responsive; consistent use of primary/accent/success colors and fonts (e.g. Patrick Hand, Nunito). Forms shall have clear labels, validation messages, and loading/error states.
- **Mobile:** Touch-friendly controls; clear navigation; audio recording and playback with permission handling.
- **Accessibility:** Forms and interactive elements shall be operable via keyboard and, where applicable, screen readers (WCAG 2.1 Level AA as target).

4.2 Hardware Interfaces

- **Server:** Standard x86_64 or ARM server (or container host) for backend and database.
- **Client:** Microphone required for pronunciation exercises; speakers/headphones for reference audio.
- **Mobile:** Device microphone and audio output; camera not required for core phonics flow.

4.3 Software Interfaces

- **Backend:** Python 3.10+, FastAPI, SQLAlchemy, asyncpg (or equivalent), Azure Cognitive Services Speech SDK.
- **Database:** PostgreSQL 12+; connection string and credentials via environment.
- **Web:** Next.js 16+, React 19, Node.js 18+; NextAuth, React Hook Form, Zod, React Query, Zustand.
- **Mobile:** Stack to be defined (e.g. React Native, Flutter, or native SDKs).

4.4 Communication Interfaces

- **API:** REST over HTTPS (HTTP allowed only in development).
- **Authentication:** Bearer token in `Authorization` header.
- **Data format:** JSON for request and response bodies.

- **CORS:** Backend shall allow configured origins (e.g. web app origin) for browser clients.
- **Azure:** Outbound HTTPS to Azure Speech (and TTS) endpoints using configured keys and regions.

5. Non-Functional Requirements

5.1 Performance

ID	Requirement	Priority
NFR-1.1	Login and token issuance shall complete within 2 seconds under normal load.	High
NFR-1.2	Exercise and phoneme list/detail API responses shall complete within 1 second for typical payload sizes.	High
NFR-1.3	Pronunciation submission (upload + assessment) may take up to 10 seconds depending on audio length and external service.	Medium
NFR-1.4	Web app first contentful paint shall be within 3 seconds on a typical broadband connection.	Medium

5.2 Security

ID	Requirement	Priority
NFR-2.1	Passwords shall be hashed with a strong algorithm (e.g. Argon2); never stored or logged in plain text.	High
NFR-2.2	API endpoints that modify data or access user-specific resources shall require a valid JWT.	High
NFR-2.3	JWTs shall have a finite expiry (e.g. configurable, default 8 days); refresh mechanism may be added later.	High
NFR-2.4	Sensitive configuration (database URL, secrets, API keys) shall be supplied via environment variables or a secure vault, not hard-coded.	High

ID	Requirement	Priority
NFR-2.5	User A shall not access or modify user B's progress, scores, or profile except where allowed by role (e.g. teacher viewing associated students).	High
NFR-2.6	File uploads (audio) shall be validated by type and size to reduce risk of abuse.	Medium

5.3 Availability and Reliability

ID	Requirement	Priority
NFR-3.1	Backend and database shall be deployable in a way that allows recovery from single-node failure (e.g. restarts, managed DB).	Medium
NFR-3.2	Dependency on Azure Speech shall not block core app usage when the feature is disabled or degraded; errors shall be handled gracefully.	Medium

5.4 Usability

ID	Requirement	Priority
NFR-4.1	The web UI shall be usable on viewports from 320px to 1920px width.	High
NFR-4.2	Error messages (validation, network, server) shall be clear and actionable where possible.	High
NFR-4.3	Learning flows (lesson → exercise → record → feedback) shall require minimal steps and clear next actions.	High

5.5 Maintainability and Scalability

ID	Requirement	Priority
NFR-5.1	Backend code shall follow a consistent structure (e.g. routers, services, CRUD, models) and use type hints.	High
NFR-5.2	Database schema changes shall be applied via versioned migrations (e.g. Alembic).	High
NFR-5.3	API versioning (e.g. /api/v1/) shall be used to allow future backward-compatible changes.	High
NFR-5.4	The system shall be deployable using containers (e.g. Docker) for backend and database.	Medium

6. System Attributes

6.1 Reliability

- Backend shall handle invalid input without crashing (validation and error responses).
- Database transactions shall be used for operations that create or update multiple related records (e.g. progress and score).

6.2 Maintainability

- Code shall be organized into modules (API, services, CRUD, models, utils).
- Configuration shall be centralized and environment-based (dev/test/prod).
- Dependencies shall be pinned or ranged in version-controlled files (requirements.txt, package.json).

6.3 Portability

- Backend shall run on common Linux distributions and, for development, on Windows/macOS.
- Web app shall run on any host that supports Node.js and the specified Node version.

7. Appendices

Appendix A: Environment Variables (Backend)

Variable	Description	Required
DATABASE_URL	PostgreSQL connection string (e.g. postgresql://user:pass@host:port/dbname)	Yes
SECRET_KEY	Secret for JWT signing	Yes
ENV_STATE	dev test prod	No (default: dev)
DEBUG	true false	No
ACCESS_TOKEN_EXPIRE_MINUTES	Time in minutes	No (default: 8 days)
AZURE_SPEECH_KEY	Azure Speech subscription key (pronunciation)	If using Speech
AZURE_SPEECH_REGION	Azure region for Speech	If using Speech
AZURE_OPENAI_*	Optional OpenAI/Azure OpenAI (if used)	No

Appendix B: API Base and Version

- **Base URL (example):** https://api.example.com or http://localhost:8000
- **API prefix:** /api/v1
- **Authentication:** Authorization: Bearer <access_token>

Appendix C: Document History

Version	Date	Author	Changes
1.0	February 2025	—	Initial SRS (backend, web, mobile; current + planned)

End of Software Requirements Specification